10/1/26

UNIT-2 Transaction flow Testing

A transaction is a unit of work seen from system user's point of view.

→ A transaction consist of a sequence of operations, some of which are performed by a system, person's or device that are outside of the system.

Transaction begins with birth i.e., they are created as results of some external points.

→ At the conclusion of the transaction processing the transaction is no longer in the system.

Examples of Transaction :-

A transaction for an online information retrival system might consisting of the following steps

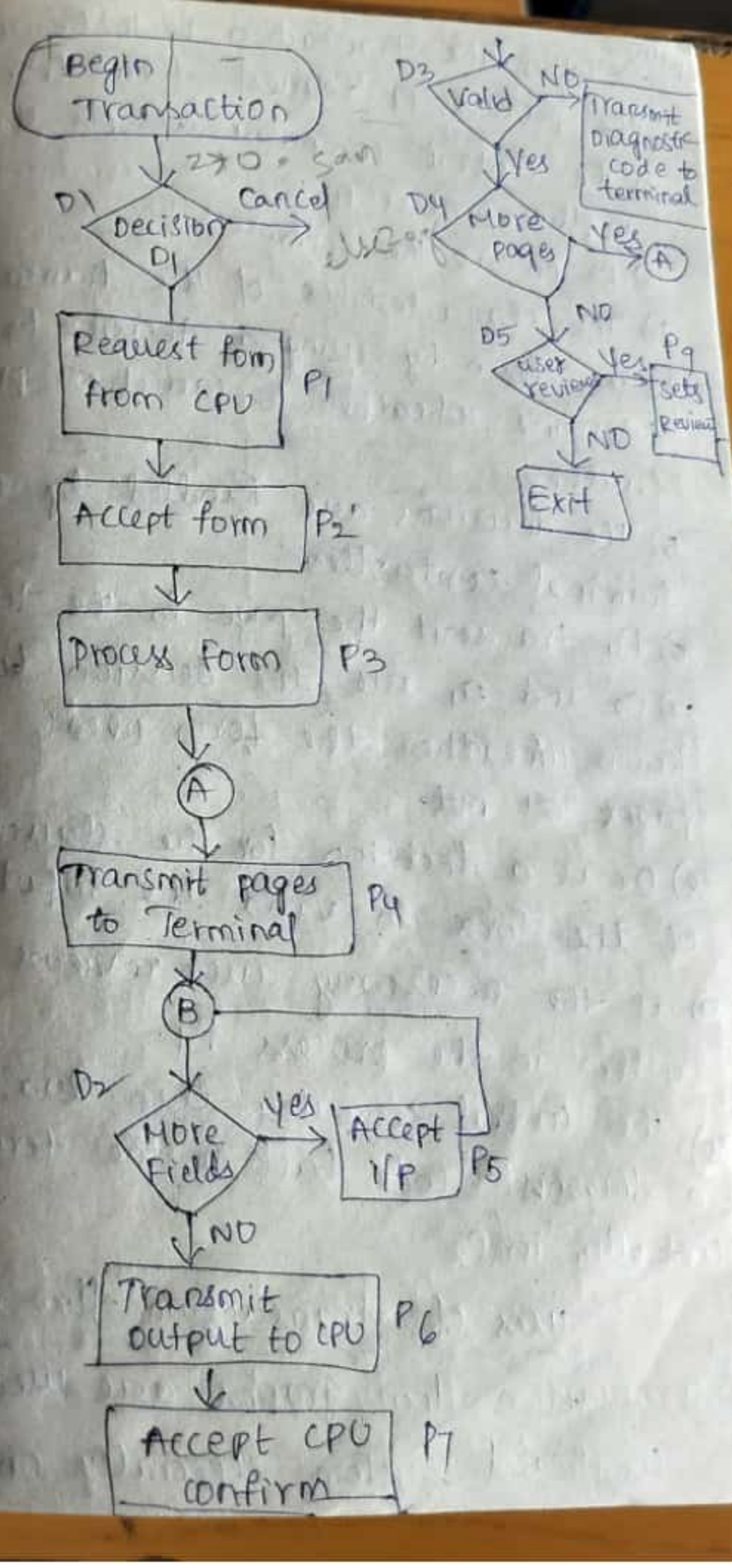
1. Accept input (tentative ~~worth~~ birth)
2. Validate input (birth)
3. Transmit acknowledgement (TO requester)
4. Do input processing
5. Search file
6. Request directions from user
7. Accept input
8. Validate input

9 process request
10 update file
11 → Transmit output
12 → Record transaction in long and clear
(death)

Transaction Flow graphs :-

→ Transaction flows are introduced as a representation of the system processing
→ The methods that we are applied to control flow graphs are then used for functional testing.
→ Transaction flows and transaction flow testing are to the Independent tester what control flows are the path system testing are to the programmer.
→ The transaction flow graph is to create a behavioural model of the program that leads to functional testing.
→ The transaction flow graph is a model of the structure of the system behavior (functionality)

An example of transaction flow is as follows (Online Information Retrival System - retrive the application from the CPU)



19/11/26

→ When the transaction is to be initia
the process P1 request form from th
2) The central computer accepts the to
in the process P2
3) P3 process the form.
4) The characteristics of the transaction
are shown by using a decision box D1
determine whether to cancel or proce
further.
5) The decisions are handled by the
terminal controller
6) P4 transmit the page to the termina
7) D2 and D4 are the decision box to
know whether the form needs More
pages or not.
8) D3 is a decision for the structure
of the form to valid the input.
9) If the necessary user reviews whol
system in the process P9.
10) The central computer then transmit
a diagonist code back to the terminal
controller in P8

uses of transaction flow graphs

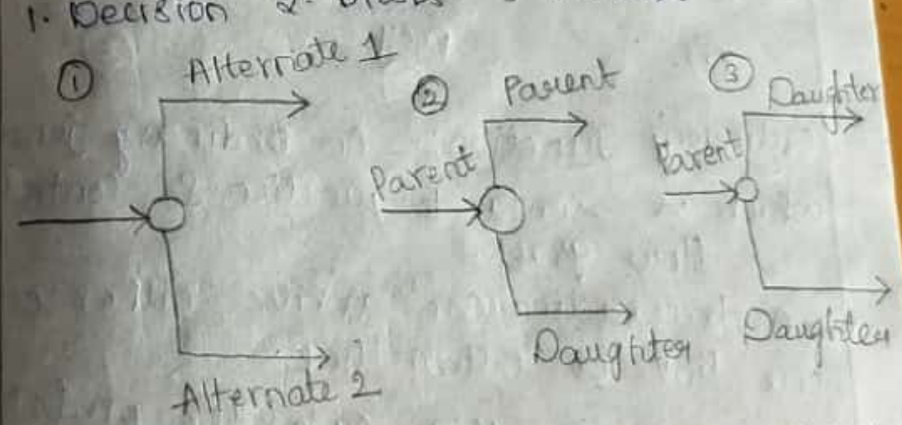
→ Transaction flow graphs are used in complicated system especially online system

→ A big system such as air traffic controller or airlines reservation system
→ The flows are represented simple flow graphs, many have a single straigh path.

The Complications

In the many systems transactions can give birth to other and transactions als merged

Births:- There are 3 different possibilities
1. Decision 2. Biosis 3. Mitosis

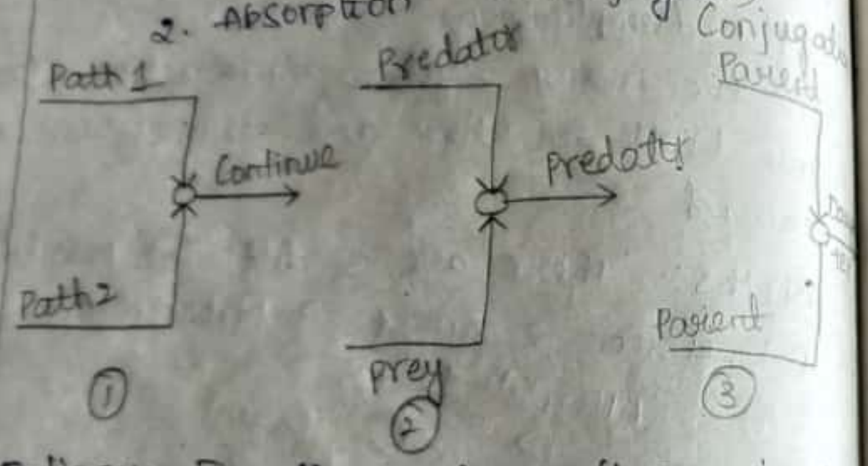


Decision:- Here the transaction will take one alternative or other alternative but not both.

Biosis:- Here the incoming transaction gives birth to a new transaction and both transactions continue on their separate paths and the parents retains it's identity

Mitosis:- Here the parent transaction is destroyed and two new transactions are created.

Merges:- They are classified into 3 types. (reverse of decision)
1. Ordinary Junction.
2. Absorption
3. (conjugation)



Ordinary Junction:- An ordinary junction which is similar to junction & control flow graph.
→ A transaction can arrive either on one link or other link.

Absorption:- In this case the predator transaction absorbs prey transaction The prey gone but the predator retains its identity

conjugation:- In this case the two parent transactions merge to form a new daughter

Transaction. Testing Techniques

How

- Complicated systems that process a lot of different, complicated transactions should have explicit representations of the transaction flows, or the equivalent.
- Transaction flows are like control flow graphs and consequently we should expect to have them in increasing levels of detail.

Inspections, Reviews and Walkthroughs

- Transaction flows are natural agenda for system reviews or inspections
- In conducting the Walkthroughs, you should:
- Discuss enough transaction types to account for 98.1. - 99.1. of the transaction the system is expected to process.
- Discuss paths through flows in functional rather than technical terms.
- Ask the designers to relate every flow to the specification and to show how the transaction, directly or indirectly, follows from the requirements.
- Make transaction flow testing the corner stone of system functional testing just as

path testing is the corner stone of unit testing.
. select additional flow paths for loops, extreme values and domain boundaries.
. Design more test cases to validate all births and deaths.
. publish and distribute the selected test paths through the transaction flows as early as possible so that they will exert the maximum beneficial effect on the project.

path selection

. select a set of covering paths (C1+C2) using the analoguous criteria you used for structural path testing.
. select a covering set of paths based on functionally sensible transactions as you would for control flow graphs.
. Try to find the most tortuous, longest, strangest path from the entry to the exit of the transaction flow.

C2 → Branch testing.

path sensitization:-

. Most of the normal paths are very easy to sensitize - 80% - 95% transaction flow coverage (C1 + C2) is usually easy to achieve.
. The remaining small percentage is often very difficult.
. sensitization is the act of defining the transaction. If there are sensitization problems on the easy paths, then bet on either a bug in transaction flows or a design bug.

path Instrumentation:-

. Instrumentation plays a bigger role in transaction flow testing than in unit path testing.
. The information of the path taken for a given transaction must be kept with that transaction and can be recorded by a control transaction dispatcher or by the individual processing Modules.
. In some systems, such traces are provided by the operating systems or a running log.

20/1/26

Inspections :- Any formal review is conducting while executing a task than it is called a inspection.
→ Inspections are done to identify errors in a program
→ We have different inspections →

1. checking syntax errors
2. refering the program
3. violating rules
4. comparsion of a code
5. Refering code
6. Comparing flow graph
7. senitization path inspection

Reviews :- Examine a project related work and the process related to that work it is called review.

→ Reviews are done to identify errors in semantic documents

→ Different reviews:

1. Review planning
2. Review zero document information
3. Preparing review meeting
4. Objective of review meeting
5. Taking a different process
6. Review scheduling.

The implementation of transaction flow

Transaction flow is usually in design of system control structure and associated with database

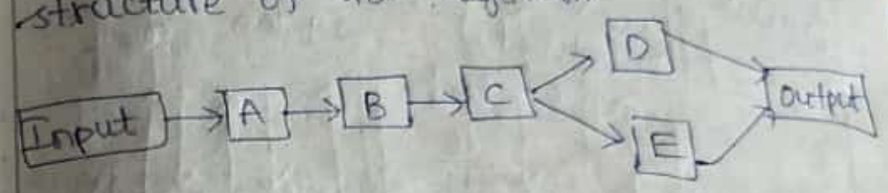
Transaction is represented by a token.

→ Transactions are placed in Transaction control box

Transactions in transaction control box are placed according to their flow

→ Transaction flow graph is graphical representation of the token.

→ Transaction flow graph is not control structure of the program but control structure of the system



The transaction go through input processing and passes through process A.

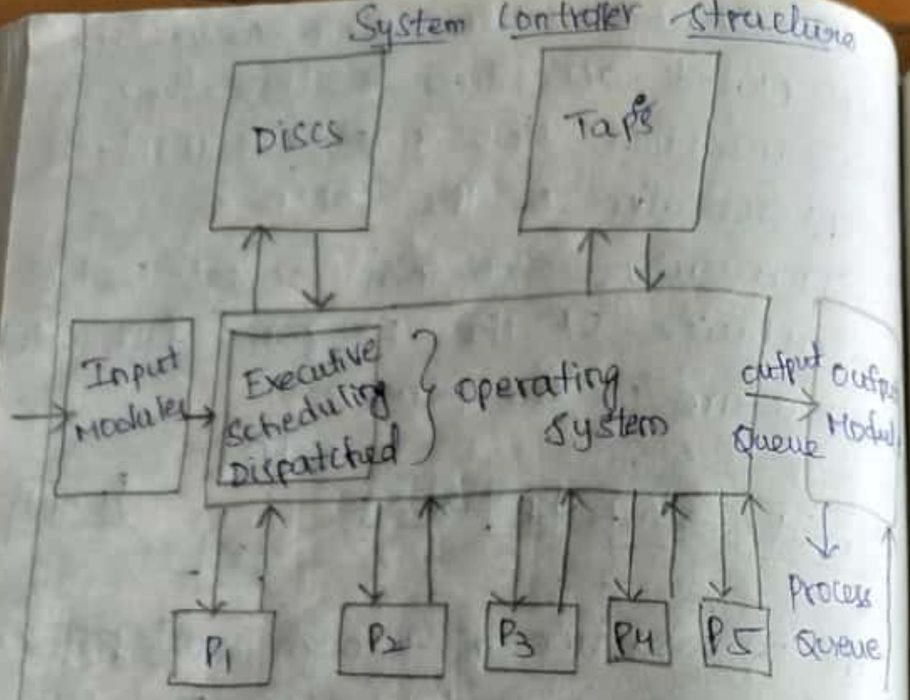
The result of A passes through process B followed by process C

→ The transaction c passes through either D, E for output processing

Operating System Transaction flows (by using system control structure)

→ Next page (figure)

System Controller Structure



Advantages & disadvantages of Transaction flow

Advantages

1. Transaction flow uses structural path testing for selecting path
2. Many bugs don't occur in this path selection.
3. The cost is paid only on some well Known transaction (online transaction
4. Path is select which is effective

Disadvantages

→ The path selected is longest path
→ The code correction is expensive
→ The bugs are caused due to the implementation errors.

2/11/26

Data Flow Testing

Basic of Data flow testing

• Data flow testing is the name given to a family of test strategies based on the selecting paths.

For example: pick enough paths to ensure that every data object has been initialized prior to use, all defined objects have been used for everything.

Data flow Machines:-

There are two types of data flow machines with different architectures.

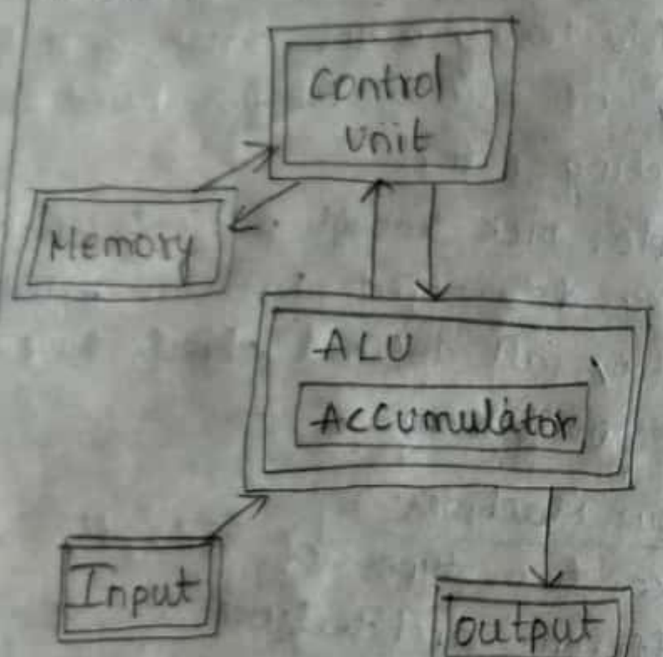
① Van - Neuman Machines
② Multi - instruction, multi-data machine (MIMD)

Van-Neuman Machines

Most of the computers today we are using van-Neuman machines

→ Van-Neuman machine architecture execute one instruction at a time in the following micro instruction sequence

1. Fetch the instruction from memo
2. Interpreter instruction
3. Fetch operands
4. process or execute
5. Store the results
6. Increment program counter.
7. Goto step 1



program control flow with von Neuman paradigm

Given m, n, p, q find e

$e = (m+n+p+q) * (m+n-p+q)$

a: = m+n
b: = p+q
c: = a+b
d: = a-b e: = c*d

Multiple representation of control flow graph can be possible

Multi-Instruction Multi Data Machine (MIMD)

→ These machines can be fetch several instructions and objects in a parallel
→ They can also do arithmetic and logical operations at a time on different data objects
→ The decision of how to sequence the depends on the objects

20/1/26 Data flow graph

The data flow graph consisting of nodes and links having arrows. These links are known as directed links

→ The data flow between the data object in the data flow graph
→ The data flow graph not only shows of the data but also show the deviation between data object to be implemented.
→ Data object state and usage
→ Data object have 3 states they are
1. Data object can be created
2. They can be killed
3. They can be used

→ Data object can be used into two different ways.
1. In calculation part
2. In control flow graph

Data flow Anamolies:-

An anomaly is denoted by a two-character sequence of actions. For example, ku means that the object is killed and then used, where as dd means that the object is defined twice without an intervening usage. What is an anomaly is depend on the application.

There are nine possible two-letter combinations for d, k and u. Some are bugs, some are suspicious and some are okay.

1. dd - probably harmless but suspicious. why define the object twice without an intervening usage?
2. dk: probably a bug. Why define the object without using it?
3. du: the normal case. The object is defined and then used.
4. kd:- normal situation. An object is killed and then redefined.
5. kk:- harmless but probably buggy. DO you want to be sure it was really killed?
6. ku:- a bug, the object does not exist.
7. ud:- usually not a bug because the language permits reassignments at almost any time.
8. uk:- normal situation
9. uu:- normal situation.

Data flow Anomaly state graph

Data flow anomaly model prescribes that an object can be in one of four distinct states:

0. K:- undefined, previously killed, does not exist
1. D:- defined but not yet used for anything.
2. U:- has been for computation or in predicate.
3. A:- Anomalous

These capital letters (K, D, U, A) denote the state of the variable and should not be confused with the program action denoted by lower case letters.

Unforgiving Data-Flow Anomaly Flow Graph

: unforgiving model, in which once a variable becomes anomalous it can

never return to a state of grace

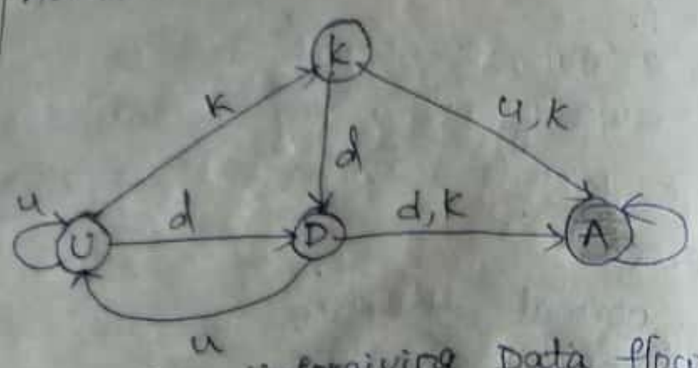


Figure 3.5: Unforgiving Data flow Anomaly state graph

Assume that the variable starts in the k state - that is, it has not been defined or does not exist. If an attempt is made to use it, or kill it (e.g., say that we're talking about opening, closing and using files and that 'killing' means closing), the object's state becomes anomalous (state A) and once it is anomalous, no action can return the variable to a working state.

If It is defined (d), it goes into the D, or defined but not yet used, state. If it has been defined (D) and refined (d) or killed without use (k), it becomes anomalous, while usage (u) brings it to the U state. If in U, redefinition (d) brings it to D, U keeps it in U, and K kills it.

Data- flow Testing - Basics

Data flow Anomalies

A two letter sequence of actions

dd: harmless, suspicious

dk: probably a bug

du: normal

kd: normal

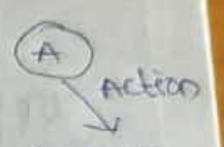
kk: harmless, but probably a bug.

ku: a bug

ud: normal Redefinition.

uk: normal

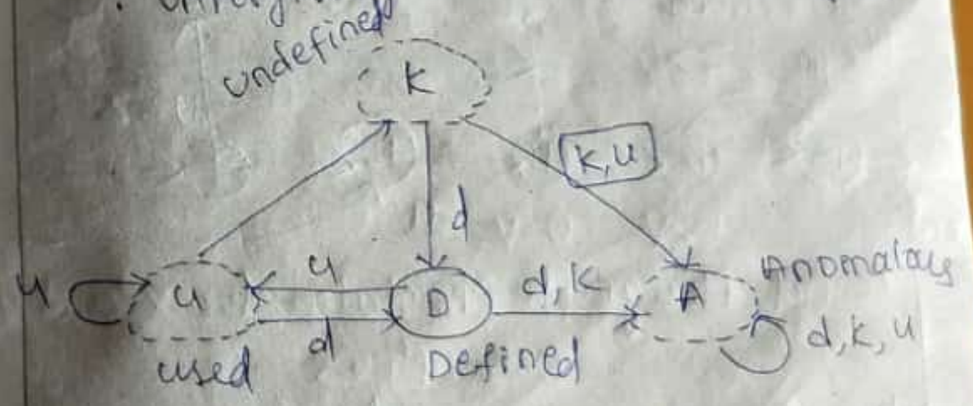
uu: normal




```
If A>0 then
   X:=1
ELSE X:=-1
A:=0
A:=0
A:=0
A:=B+C
```

Data flow Anomaly state graph

- Object state
- Unforgiving data flow state graph



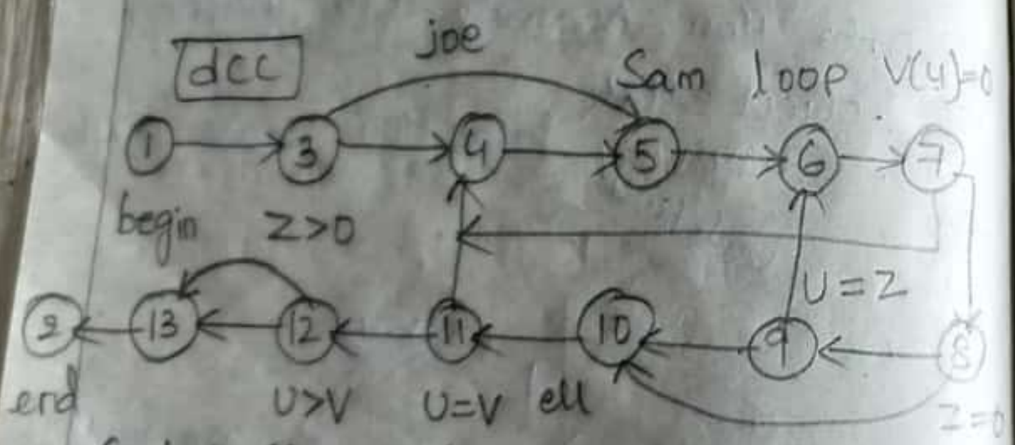
Data flow Testing Example

Step 1: Input X, Y; Z = X+Y, V = X-Y

Step 3: if Z>0 goto Sam

Step 4: Joe: Z=Z-1

step 5: Sam: z=z+v
step 6: for u=0 to z
step 7: v(u), u(v)=(z+v)*u
if v(u)≠0 goto joe
z=z-1
step 8: if z=0 goto E11
step 9: u=u+1
if (u=z) goto (step 6)
step 10: E11: v(u+u(v))=u+v
step 11: if u=v goto joe
step 12: if u>v then u=z
step 13: z=u
step 2: end



Control flowgraph Annotated for x and y

Data flows

23/11/26

DU Define Node

step 1: Input x, y; z=x+y, v=x-y
step 3: if z>0 goto Sam
step 4: joe: z=z-1

step 5: Sam: z=z+v
step 6: for u=0 to z
step 7: v(u), u(v)=(z+v)*u
if v(u)≠0 goto Joe
z=z-1
step 8: if z=0 goto E11
step 9: u=u+1
if (u=z) goto (step 6)
step 10: E11: v(u+u(v))=u+v
step 11: if u=v goto joe
step 12: if u>v then u=z
step 13: z=u
step 2: end

(Diagram same)

DU use Node

step 1: Input x, y; z=x+y, v=x-y
step 3: if z>0 goto Sam
step 4: joe: z=z-1
step 5: Sam: z=z+v
step 6: for u=0 to z
step 7: v(u), u(v)=(z+v)*u
if v(u)≠0 goto joe
z=z-1
step 8: if z=0 goto E11
step 9: u=u+1, if (u=z) goto (step 6)

step 10: E11: v(u+u(v))=u+v
step 11: if u=v goto joe

Step 12: if U>V then U = Z
Step 13: Z = U
Step 2: end

Strategies of Data flow testing

→ Data Flow testing strategies are structural strategies
→ Data Flow Testing strategies are based on selecting path segments (also called sub paths) that satisfy some characteristics of data flows for all data objects.
→ In constrast to the path-testing strategies, data-flow strategies take into account what happens to data objects on the links in addition to the raw connectivity of the graphs.
→ In other words, data flow strategies require data flow link weights (d, k, u, c, p).

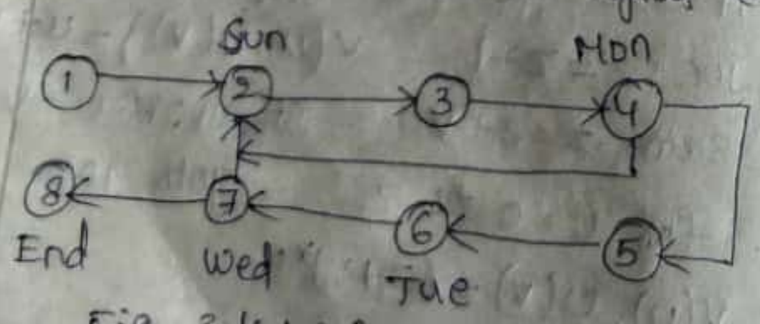


Fig. 3.4.1: Control flow graph

Terminology

1. Definition clear path segment

→ A path segment is a sequence of connected links between nodes. The first link of the path is defined and the subsequent link of that path is killed.
→ The path (1,2), (3,4), (5,6) are definition-clear. The path (1,2,3,4,5,6) is not definition-clear because the variable is defined on (2,3), (4,5) and again on (6,7)

2. Loop free path segment

→ The path which doesn't have any loop and every node of which is visited atmost once is known as a Loop-free path segment.
→ The path (2,3,4,5,6) are loop-free but the path (6,7,2,3,4,5,6,7) is not because the node 6 and 7 are visited twice and hence form a loop in that path.

3. Simple path segment

→ The path (4,2,3,4) is a simple path segment because the node 4 is visited twice in the path.
→ Hence a simple path segment is a path segment in which atmost one node of it is visited twice.
→ Hence this path can be either loop-free or can have a loop.

4. DU path

→ The path segment from node to c
that if the last link has a computat
use of X, then this path is considere
as a simple and Definition - clear pa
→ The path (b, c, e, f, a, g) such that
link (a, g) has a predicate use. Then th
path from b to a is loop-free and
definition clear.

Strategies

→ The structural test strategies discus
below are based on the program's contr
flowgraph.
→ They differ in the extent to which predi
uses and/or computational uses of
variables are included in the test s

1. All-du paths (ADUP):

A du path is a path which has a define
variable and a computational use or a
predicate use of that variable.

DEF:- All du paths strategy is a dat
flow testing that requires every definit
of every variable from every du path
to use every variable of that definitio

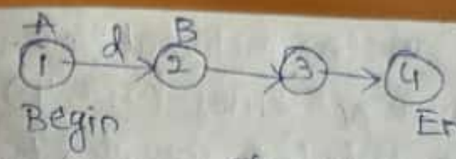


Fig. 3.4.2 Flowgraph Annotated for variables A and B

In Fig. 3.4.2, variable A and B are used on the data flow link (1,3). Any test that starts from the link (1,3) satisfies this criterion (All du paths strategy) only for variables A and B but not for all the variables.

2. All-uses (AU) strategy:-

All uses strategy helps in reducing the number of test cases.

DEF:- All uses (AU) strategy is a data flow testing strategy that requires every definition from atleast one path segment to use every variable that can be reached by that definition.

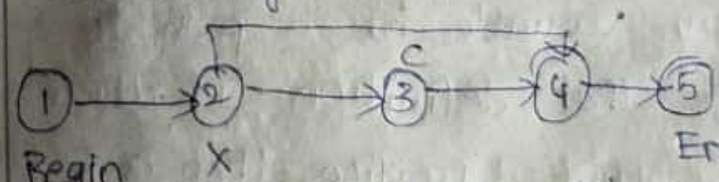


Fig. 3.4.3: Flowgraph Annotated for Variable x.

If a variable x has a predicate use in ADUP strategy, the sub-paths (2,3,4)

and (2,4) can be included in some te
In AU strategy either (2,3,4) or (2,4
can be included, but we don't have t
use both to begin a path. The link (2,
can be included in some test cases
because that's the only way to reach
the c (computational) use in that link

3. All P-uses or some C-uses (APU+C strategy)

If a variable has a predicate use then there is no need to select a computational use.

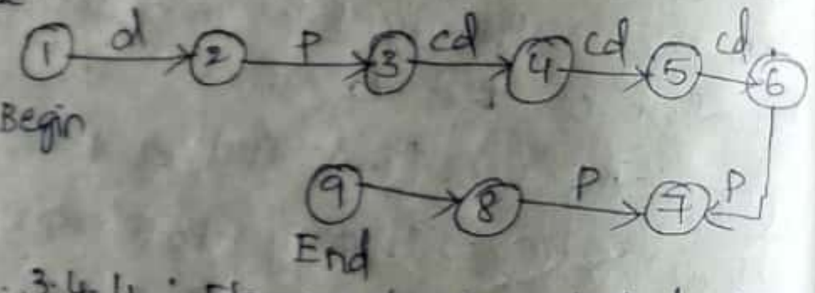


Fig. 3.4.4 : Flowgraph Annotated for Variable Z

In APU + C strategy, every definition of every variable has a path to every P-use of that definition. If there's no P-use of that definition, then a C-use is considered. In the Fig. 3.4.4 links (1,2), (3,4), (4,5), (5,6) must be included in the test cases because they contain definitions for variable Z. The links (2,3) (6,7) (8,9) must be included in test cases because they contain predicate uses of Z. In this testing strategy, if the definitions of variables d are not covered then additional computation uses ad are required

4. All C-uses or some P-uses (ACU+P strategy)

In this testing strategy first ensure that every definition has a computational use of that definition has a computational use of that definition.

DEF:- ACU+P is a strategy that requires every definition of every variable included atleast or

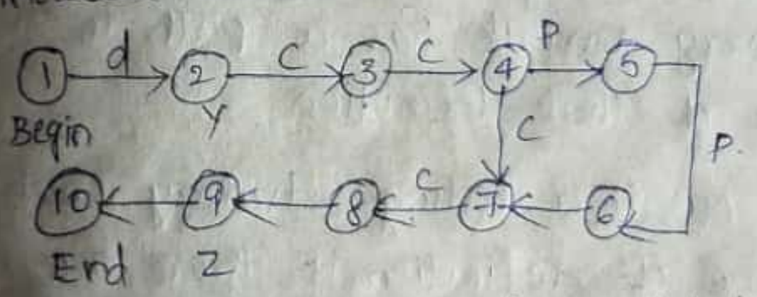


Fig 3.4.5 : Flowgraph Annotated for Variable Y

In the fig 3.4.5, ACU+P coverage is achieved for Y by Path (1,2,3,4,5,6,7,8,9,10). But several definitions of Predicate

use are not covered, that is the p (4,5,6,7). In this testing strategy every definition of every variable a path to every c-use of that definition.

5. All Definitions (AD) strategy:-

AD strategy is weaker than both ACU+P and APU+C strategies.

DEF:- AD is a strategy that requires every definition of every variable to cover atleast one of that variable. That use can be a computation use C or a predicate use P

6. All predicate-uses (APU) strategy:-

In this testing strategy every definition of every variable has a path to every p-use of that definition

DEF: APU is a testing strategy that requires a p-use of definition if there are no c-use following that definition.

ordering the strategies

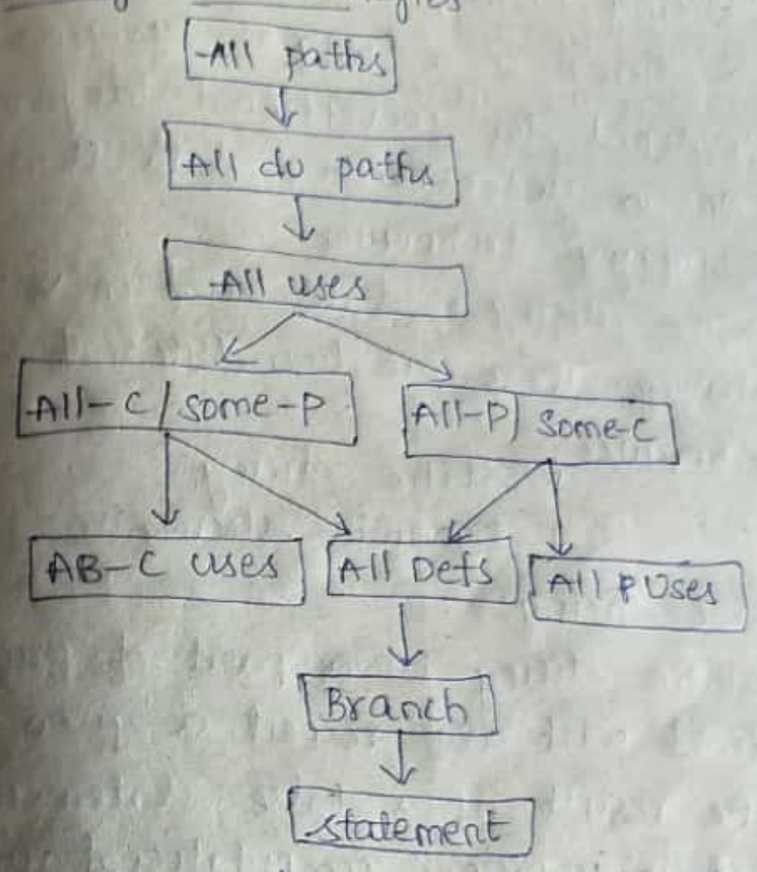


Fig 3.4.6: order of strategies based on their relative strength.

7. All-computational-uses (ACU) strategy

DEF:- ACU is a testing strategy that requires a c-use of definition if there are not p-use following that definition Therefore from definition of this strategy, ACU strategy should be weaker than ACU+P and APU+C strategies.

Slicing, Dicing, data flow and debugging

→ slicing is a program originally developed for conventional languages helps in understanding data flow and debugging techniques.

→ The slicing is done based on variable sharing. There are two types of slicing techniques

1. static slicing
2. Dynamic slicing.

Static slicing:-

→ static slicing is a part of program defined with ~~reg~~ respect a given variable

→ The result of a false statement effect in improper computational use or predicate use of some other variable.

Dynamic slicing:-

→ Dynamic slicing is a refinement of static slicing. It compares the data flow relationship with respect to static data flows.

→ Debugging tools supports slicing methods and experimented on them.

Applications

Data flow testing is used to detect the different abnormalities that may arise due to data flow Anomalies.

→ Data flow testing shows the relationship between the data objects that represents data.

→ Dataflow testing strategies helps in determining the usage of variables that are included in the test set.

→ Data flow testing is cost effective.

→ Data flow testing solve the problem that are encountered while performing.

Static and dynamic data flow Anomalies

Static Analysis for Data flow Anomaly Detection	Dynamic Analysis for data flow Anomaly detection
1) static analysis is done at compile time in data flow anomaly detection	1) Dynamic analysis is done at run time in data flow anomaly detection
2) static analysis is based on the source code before execution.	2) Dynamic analysis is based on the inter mediate values that result from the program execution.

3) static analysis operates by building a model of the program state, then determining how the program reacts

4) There's no way to validate the pointer values in static analysis

5) static analysis for data flow anomaly detection is more compact and faster than dynamic analysis

6) static analysis is better than dynamic analysis in language processors such as data flow anomaly detection.

7) static analysis is less precise because it usually use an abstracted model of information.

8) Applications that require precise inputs are unable to use the results of a typical static analysis

9) Source code syntax error detection is the example for static analysis result.

3) Dynamic analysis operates by executing program and observing the executions.

4) Array pointers are dynamically calculated

5) Dynamic analysis can be as fast as program execution.

6) Dynamic analysis helps in testing by detecting anomalous situations.

7) Dynamic analysis is more precise because no abstraction or approximation need to be done i.e., the analysis can examine the actual runtime behaviour of the program.

8) Applications that require correct inputs are unable to use the results of a typical dynamic analysis.

9) A division by zero warning is the example for dynamic analysis.

29/1/26

Domain Testing

specific area

→ Domains and paths
→ Nice and ugly domains
→ Domain testing.
→ Domains and interface testing.
→ Domains and testability.

• THE MODEL:- The following figure is a schematic representation of domain testing

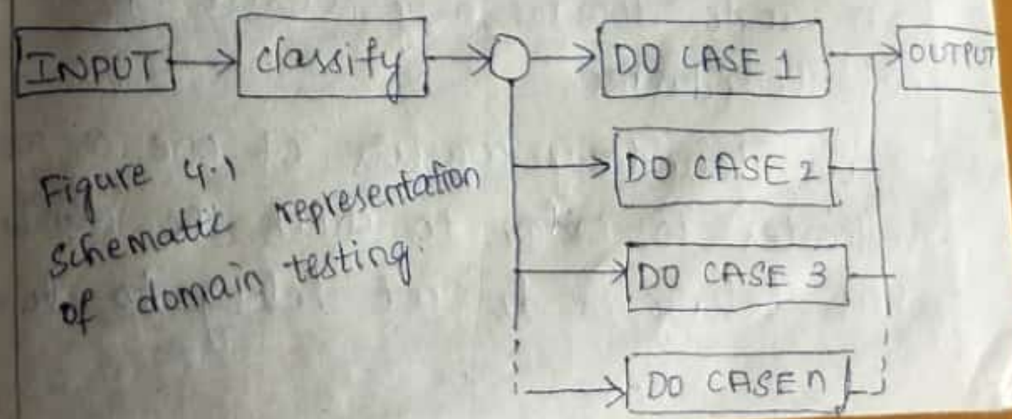


Figure 4.1
Schematic representation of domain testing

. Before doing whatever it does, a must classify the input and set it on the right path
. An invalid input (e.g.; value too big) just a special processing case called
. The input then passes to a hypothe subroutine rather than on calculations
. In domain testing, we focus on the classification aspect of the routine rat than on the calculations.
. structural knowledge is not needed this model - only a consistent, complete specification of input values for each . we can infer that for each case there must be atleast one path to process the case.

Domains and paths

Domains

→ Domains can be specified directly and are predefined by their boundaries.
→ A single domain may consist of one or more boundaries.
→ Every boundary consists of one or more predicates in order to check the numbers which belong or do not belong to domain

each domain there may be two possibilities which are as follows -es,
. Domain consists of a connected part
. Domain consists of a disconnected part

DOMAIN CLOSURE

→ The aim of the domain closure is to allow the wrong closure bugs to become as frequent domain bugs.
→ To understand domain closure, let us take an example which consists of three cases for one-dimensional domain.
→ If the boundary points are related to other domain then the boundary is said to be an open boundary.
→ whereas if the boundary points are related to that domain itself then the boundary is said to be a closed boundary.

The cases for one-dimensional domain are as follows.
a) Domain is open on one side
b) Domain is closed on both sides
c) Domain is open on both sides.
*) Domain is open on one side



Fig 4.1.1: one side open

As shown in the fig 4.1.1 there are three domains namely Domain A, Domain B and Domain C. The two circles (An empty circle which denotes an open boundary P_1 and a filled circle which denotes closed boundary P_2) Domain B is open on the point P_1 and is closed on the point P_2.

b) Domain is closed on both sides

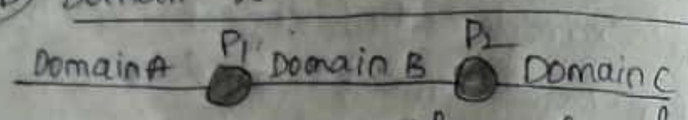


Fig. 4.1.2 Both sides closed

The Fig 4.1.2 shows that the Domain B is closed at both sides i.e., P_1 and P_2.

c) Domain is open on both sides

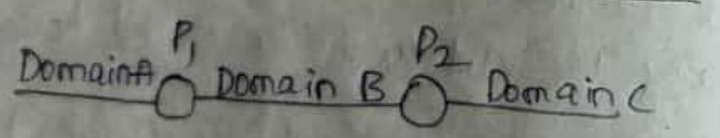


Fig 4.1.3: Both sides open.

This is the third case for one-dimensional domain

It is open on both P_1 and P_2.

Domain Dimensionality

→ Every input variable adds one dimension to the domain.

→ one variable defines domains on a number line.

→ Two variables define planar domains.

→ Three variables define solid domains.

Bug-Assumptions for Domain Testing

Ambiguous Domains:-

In this, the union of the specific domains results either in missing domains or holes. Hence, the union operation is incomplete.

This bug will occur when the combination of required domains are not completed (i.e., may be lost domains).

overloaded Domains:-

This bug will occur when the domain consists of too many conditions which results in a Null Domain.

Closure Bug:- This bug happens when we have selected the wrong predicate such as $x >= 0$ is written as $x <= 0$.

Boundary Errors:-

This bugs occurs when the boundary is shifted or when the boundary is tilted or missed.

Faulty Logic:- This bug happens when there are incorrect manipulations calculations or simplifications in a domain.

Restrictions

→ Domain has restrictions as do other testing techniques
→ They are to be productive because they may not reveal bugs.
→ In testing there are no invalid tests only unproductive tests.

Simple Domain Boundaries and Compound predicates

→ As we know, boundary is simply defined as an area of coverage in which decision on a program is predicate
→ The two predicates used for defining a boundary are simple domain boundary and compound predicate
→ Mostly simple predicate is used to define each boundary of a program in order to avoid inconsistent boundary closures. Whereas compound predicates are used in which each part of the predicate specifies a separate boundary.

Linear Vector Space

→ A linear predicate is defined by a linear inequality, using only the simple relational operators $>, >=, =, <=, <>, <$.
→ A more general assumption is that boundaries can be embedded in a linear vector space.
→ In real testing asking for a linear vector space boils down to simple linear predicates.

Loop-Free Software

→ Loops are problematic for domain testing.
→ The trouble with loops is that each iteration can result in a different predicate expression.
→ If a loop is definite, then domain testing may be useful for the processing within the loop, and loop testing

27/11/26

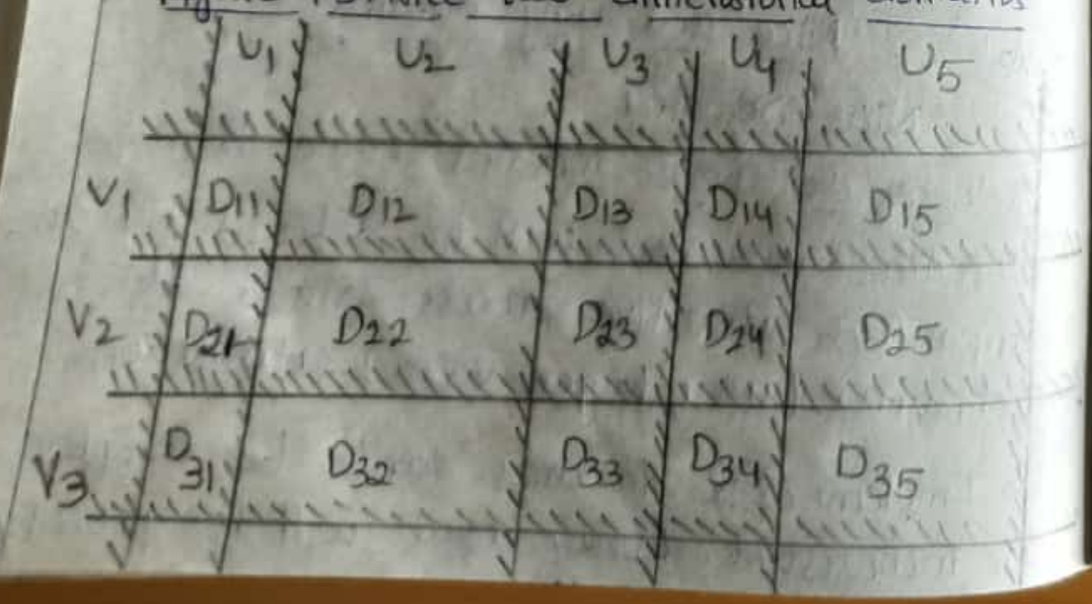
Nice and ugly domains

Nice Domains

Where do these domains come from?
Domains are and will be defined by an imperfect iterative process aimed at achieving (user, buyer, voter) satisfaction.
→ Implemented domains can't be complete or inconsistent. Every input will be processed

(rejection is a process), possibly forever. Inconsistent domains will be made consis-
→ Conversely, specified domains can be incomplete and/or inconsistent. Incomplete in this context means that there are input vectors for which no path is specified, and inconsistent means that there are atleast two contradictory specifications over the same segment of the input space.
→ Some important properities of nice domains are: Linear, complete, systematic and orthogonal, consistently closed, convex and simply connected.
→ To the extent that domains have these properities domain testing is as easy as testing gets.
→ The bug frequency is lesser for nice domains than for ugly domains.

Figure 4.3: Nice two-dimensional domains



Nice and ugly domains

Nice domains

→ There are some properities related to domain boundaries.
→ If we apply these properities then it makes domain testing very simple.
→ If we don't apply these properities then it makes domain testing very difficult.

properities

Linear and Non-Linear Boundaries

→ Nice domain boundaries can be described by using linear inequalities or equations.
→ Linear boundaries are more frequently used than the non-linear boundaries.
→ We can linearize the non-linear boundaries by using simple transformations.

Complete Boundaries

→ Nice domain boundaries have complete boundaries because they cover from plus infinity to minus infinity.
→ one of the main advantage of a complete boundary is that it requires only set of tests to verify the boundary irrespective of the number of surrounding domains.

Orthogonal Boundaries

Table 4.2.1 : Nice Two Dimensional Domains

A/B	A1	A2	A3	A4	A5
B_1	Dom 1	Dom 2	Dom 3	Dom 4	Dom 5
B_2	Dom 11	Dom 12	Dom 13	Dom 14	Dom 15
B3	Dom 21	Dom 22	Dom 23	Dom 24	Dom 25
B4	Dom 31	Dom 32	Dom 33	Dom 34	Dom 35

In the table 4.2.1, A and B are the two boundary sets that are orthogonal to each other i.e., the inequalities of A and B are perpendicular to each others. As two boundaries A and B are orthogonal they can be tested independently of one another.

In the table 4.2.1 we can see that there are 7 boundaries in A and five boundaries in B. Therefore the total number of boundaries are 7+5=12, we can denote these boundaries by using a notation O(n). If we want to tilt the above orthogonal boundary, we can do it by testing the intersection points but this can change the linear growth, O(n) is the quadratic growth $O(n^2)$.

Consistent closure

→ Consistent closures are the most simple and fundamental closures that gives a systematic results after performing the required analysis.

→ In the above figure the shading lines on the boundary shows that the domain consists of shading lines.

→ Inconsistent closures in constrast to consistent closures, are not fundamental and does not give any systematic results.

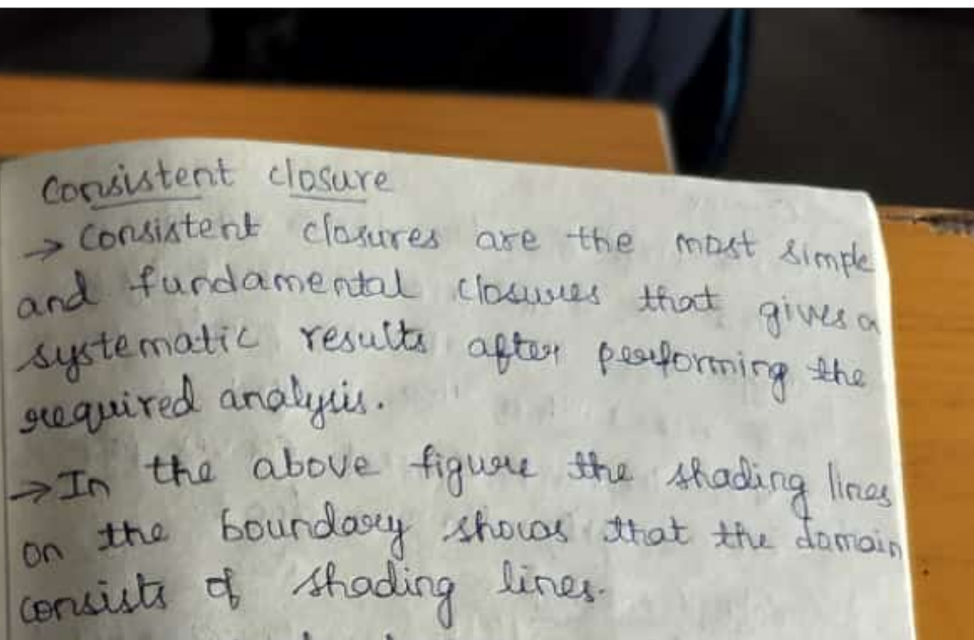


Fig 4.2.1 : Boundary closures are convex

Convex

→ For any two boundaries, with two points placed on them are combined by using a single line then all the points on the line are within the range.

→ Nice domains support convex property where as dirty domains don't.

Simply Connected

→ Nice domains are simply connected because they are available at one place as a whole but not dispersed in other domains.

→ Simply connected and Conver domains are related to each other.

Systematic Boundaries

→ Systematic boundaries refer to boundary inequalities associated with simple mathematical functions such as a constant.

→ If the boundaries are systematic then generating a test for one boundary can automatically generate the tests for the remaining boundaries within the set.

Ugly domains

→ These simplifications can be either good or bad depending upon specifications.

→ If the domains complexity is important and the programmers simplification is safe, then it will not cause any bugs.

→ When the domain complexity is important and the programmer's simplification is not safe, it will results in bugs.

Non-Linear Boundaries we don't provide values

→ These boundaries may or may not be present in simple programming because these boundaries do not provide an information regarding simplification of boundaries even if they important.

- Ambiguity and contradictions:

programs can't be ambiguous contradictory whereas specifications can be.

→ It leads to holes or cracks in the input space. A hole in the one-variable input space can be easily viewed whereas ambiguity in two variables input space is difficult to identify.

Ambiguities and Contradictions

- Domain ambiguities are holes in the input space.
- The holes may lie within the domains or in Cracks between domains.

- Two kinds of contradictions are overlapped domain specifications and overlapped closure specifications.
- Figure 4.7c shows overlapped domain and figure 4.7d shows dual closure assignment.

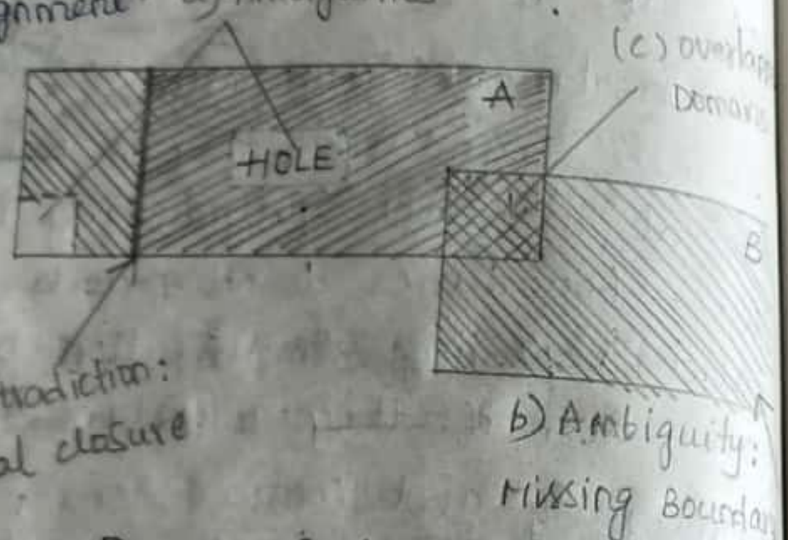


Figure 4.7: Domain Ambiguities and contradictions.

Simplifying the topology

- The programmer's and tester's reaction to complex domains is the same - simplify.
- There are three generic cases: concavities, holes and disconnected pieces.
- programmers introduce bugs and testers misdesign test cases by: smoothing out concavities (Figure 4.8 a), filling in holes (Figure 4.8 b) and joining disconnected pieces (Figure 4.8 c)

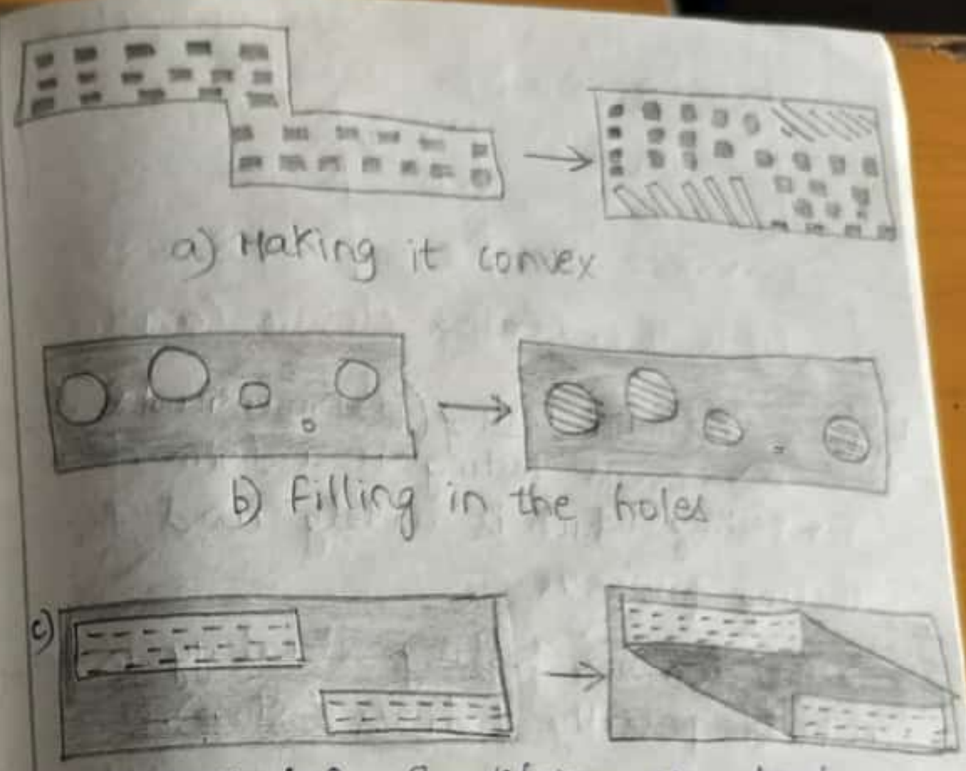
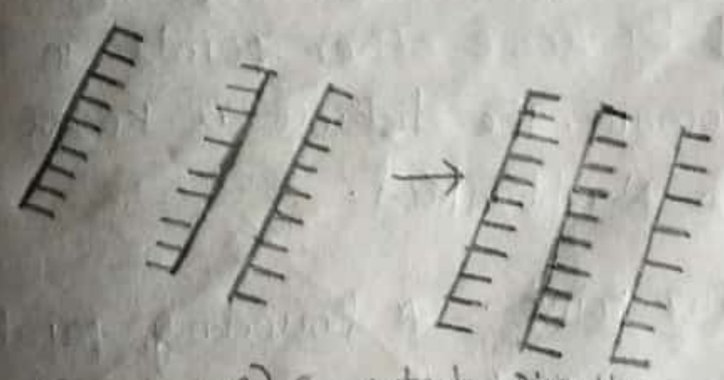


Figure 4.8: Simplifying the topology

Rectifying Boundary Closures

- If domain boundaries are parallel but have closures that go every which way (left, right, left ...) the natural reaction is to make closures go the same way (see figure 4.9)



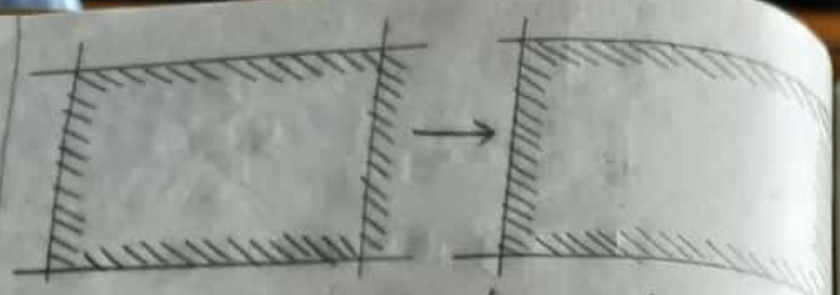


Figure 4.9: Forcing closure Consistency

Date 27/11/26

Domain Testing (1-Dimensional & 2-D

→ Domain testing is a form of wh box testing which is widely used for testing purposes.

→ The domain-testing strategy is sim although possibly tedious (slow).

→ classify what can go wrong with boundaries, then define a test strategy for each case. Pick enough points to test for all recognized kinds of boundary errors.

Interior point

An interior point in a domain can be defined as a point which specifies certain discovered by some other points in the same domain. The distance is known as an Epsilon Neighbourhood.

Boundary point :- A boundary point refers to a particular point such that within a specific epsilon neighbourhood, both the points belonging to and not belonging to the domain exists.

Extreme point :- A point that lies between two specific points of a domain but not between any two other arbitrary points is known as "Extreme points"

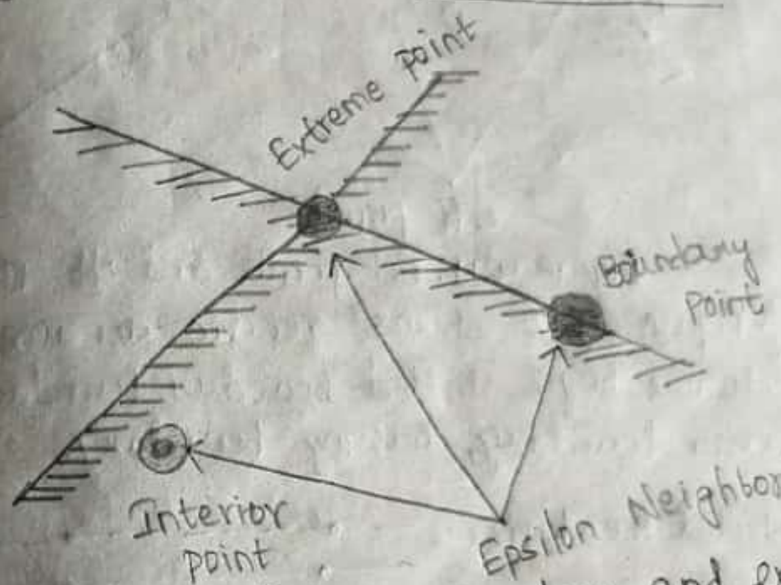


figure 4.10 : Interior, Boundary and extreme points

→ An on point is a point on the boundary.

→ If the domain boundary is closed, and off point is a point near the boundary but in the adjacent domain.

→ If the boundary is open, and off point is a point near the boundary but in the domain being tested; see figure 4.11. You can remember

this by the acronym COOOOOI : closed outside, open off inside.

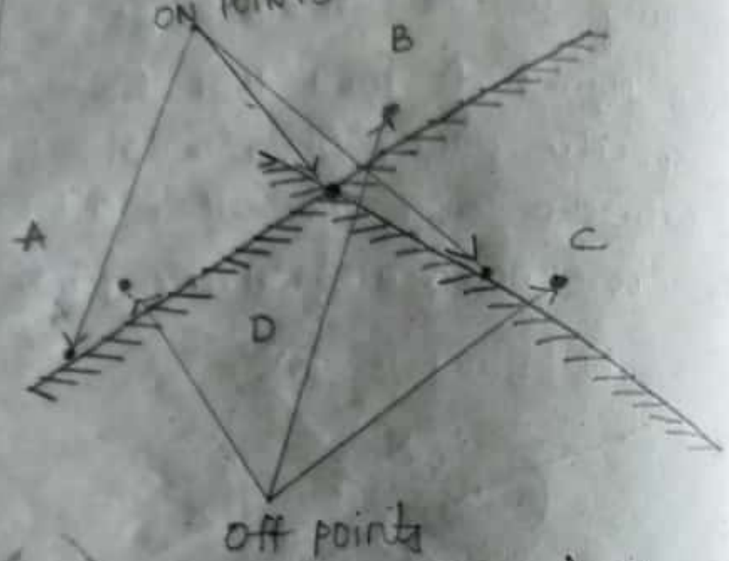


Figure 4.11: on points and off points

· Figure 4.12 shows generic domain bugs closure bug, shifted boundar boundaries, extra boundary, missing boundary.

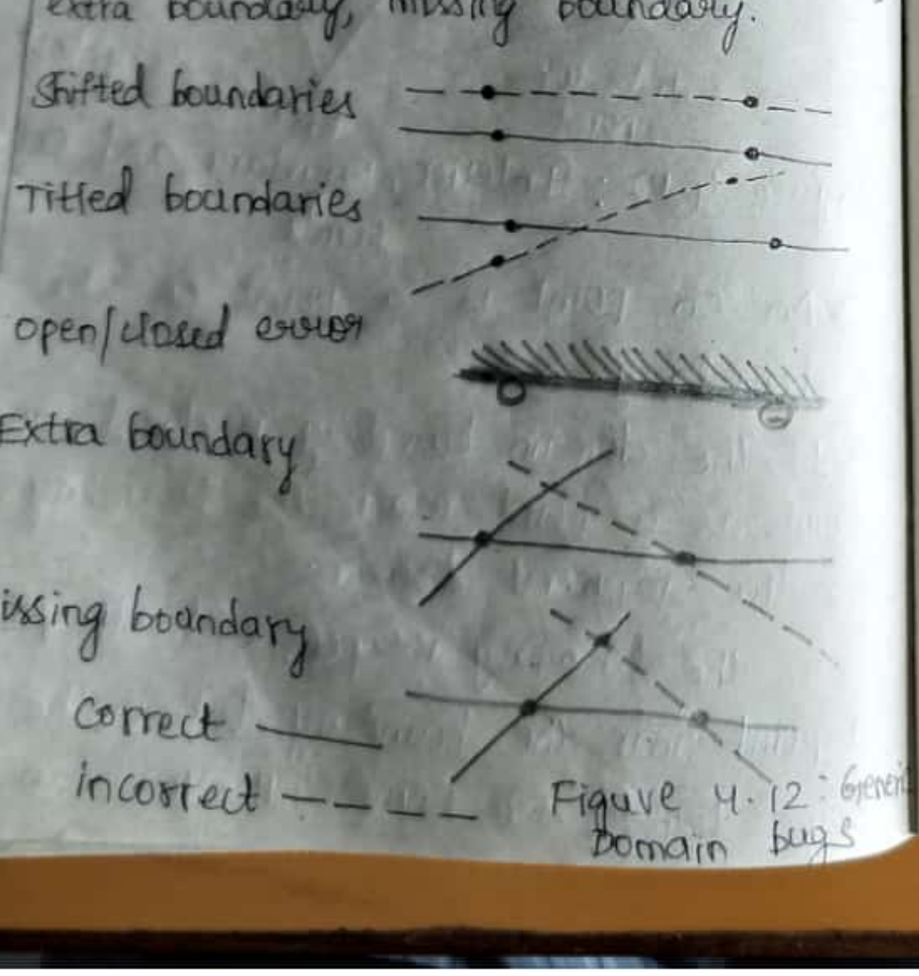


Figure 4.12: Generic Domain bugs

Testing boundary bugs for one-dimensional domains

The following are the common domain boundary bugs that occur in one-dimensional domains:

For determining various types of bugs, we need to consider two domain boundaries i.e.) one-dimensional open domain boundary and one-dimensional closed boundary.

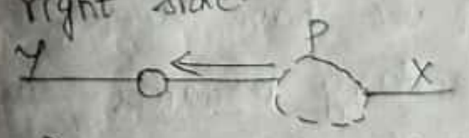
1. One-dimensional open domain boundary:-

An open domain boundary can be represented as an small empty circle. Various domain bugs are as follows:

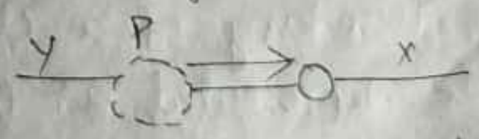


Fig 4.3.3: closure Bug

b) Boundary shifting:- This bug happens when a boundary is shifted either left side or right side.



a) Boundary is shifted to the left side



b) Boundary is shifted to the right side

Figure 4.3.4: Boundary shifting.

c) Extra Boundary:- This bug occurs when an extra point is added for instance 'z' is added as shown in fig. we can detect this extra boundary by observing two domain boundaries

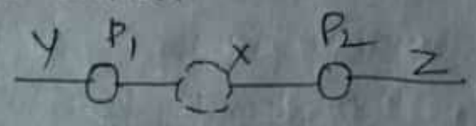


Fig 4.3.5: Extra boundary

d) Missing Boundary:- This bug occurs when an extra point is added for instance 'z' is added as shown in fig we can detect this extra boundary by observing two domain boundaries



Fig 4.3.6: Boundary Missing

2. One-dimensional closed domain boundary

The bugs which we have seen in the open domain boundary remains same as in closed domain boundary except that, the point or boundary of a closed domain can be represented as filled circle and it requires a strategy known as closed off outside.

Testing the boundary bugs for two-dimensional domains

Let us consider that x and y are two neighbouring domains such that the boundary is opened with respect to y (domain) and is closed relative to x. Now we will see the first case i.e., the boundary is open with respect to y domain

A) Closure bug:- It depicts a wrong closure, because it uses some wrong or incorrect operators.

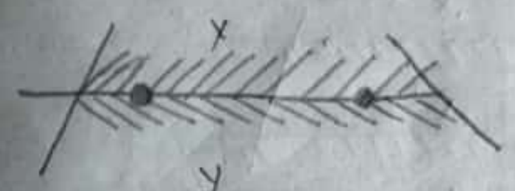


Fig 4.3.7: Closure Bug.

b) Titled Boundary:- This boundary results when the coefficients of inequality are incorrectly taken in the boundary. Suppose if the actual boundary is $2a+4b > 13$ and if $4a+2b > 13$ is used instead, then it results in titled boundary.

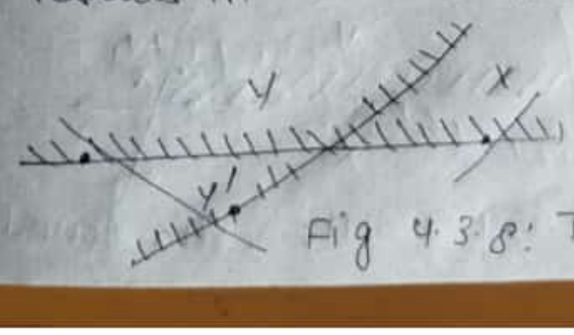


Fig 4.3.8: Titled boundary

c) Shifted up boundary:- The boundary is shifted up, which transfers some portion of the domain x into y, and this shifted up boundary is denoted by y. If the boundary a+b<=4 is used instead of a+b<=40 then it results to shifted up boundary.

d) Shifted down boundary:- This case is reverse of shifted up boundary. But the bug can be caught by the two on points as depicted.

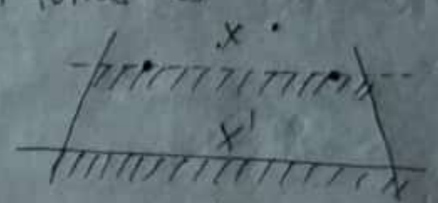


Figure 4.3.10: Shifted down boundary

e) Extra boundary:- An extra predicate is added to build an extra boundary. This extra boundary passes through several domains and leads to many test failures for the same bug.

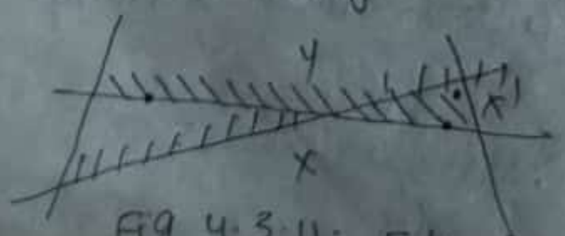


Fig 4.3.11: Extra Boundary

f) Missing Boundary:- This is one of the possible cases for closed boundaries in which a boundary is generated by leaving out the predicate. This boundary combines various boundary domains and may effect large number of test failures even though there is only one bug.

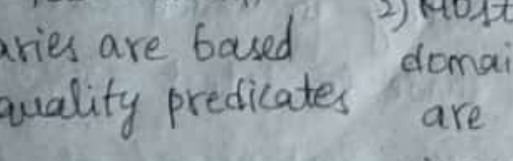


Fig 4.3.12: Missing Boundary

Differences between equality and inequality predicates

Equality predicate	Inequality predicate
1) Equality predicates can be defined by an equality equation such as x+y=10.	1) Inequality predicates can be defined by an inequality equation, such as x+y>10 or x+y<10.
2) only few domain boundaries are based on equality predicates	2) Most of the domain boundaries are based on inequality predicates

Random Testing

→ Random testing is a form of functional testing that is useful when the time

needed to write and run directed tests are too long.
→ while doing random testing you must ensure that they cover the specifications
→ It is often correct that random testing is less efficient than direct testing.

Date 29/1/26

Domain and Interface Testing

→ Domain Testing plays a very important role in integration testing.
→ It is a technique used to determine wheather the components are accurate or not.

The possible form of interface are as follows:-

→ Global variable of two components
→ Data objects of two components
→ Sub routine call of the two components

Interface testing can be defined as a method of testing in which call to sub routine is made to detect errors.
→ some of the applications of domain testing for interface testing are as follows

1. Domains & range.
2. Closure compactibility
3. Span compactibility
4. Interface range and domain compactability.

- Caller domain → Caller range (unit test)
- caller range → Called domain (Integration testing)
- Called domain → Called range (unit testing)

The domains and Range

Domains are nothing but the value taken by the input as function.
→ Range means it shows the boundaries
→ Ranges are the value obtained as the output from the function.
→ In the domain testing we are focused on input values but the interface testing mainly focus on o/p values.
→ Interface testing is also used to check the accuracy between the domain and the range.

The closure Compactability

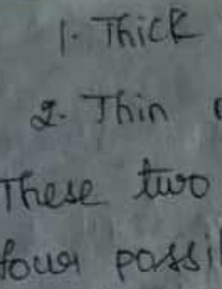
→ In the closure compactability domain lies between the numbers 1-18.
→ The closure compactability can be determined by taking a type of no. lines they are as follows.

1. Thick number line (closed line)
2. Thin number line (open line)

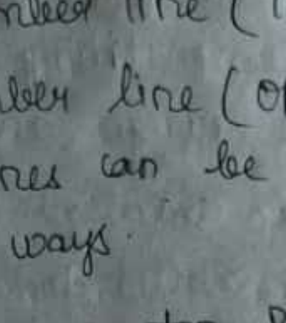
These two lines can be represented four possible ways.

1. Thick lines on top & bottom
2. Thin line on top, thick line on bottom
3. Thick line on top, thin line at the bottom.
4. Thin lines on top and bottom

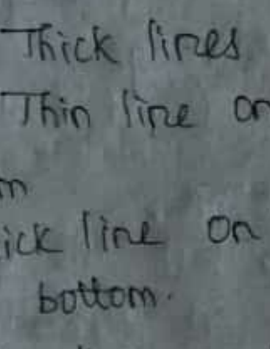
(i)



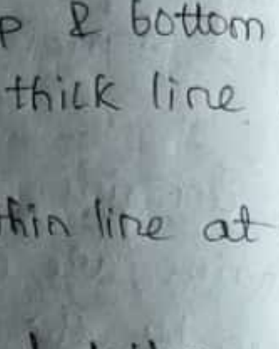
(ii)



iii)



iv)

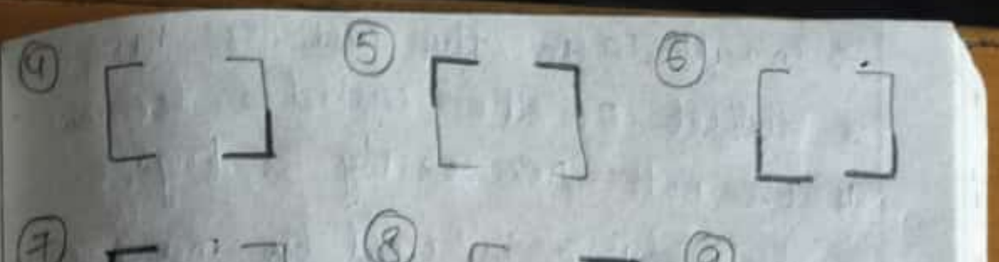


In closure compactability we have 4 possibilities

T L [BL

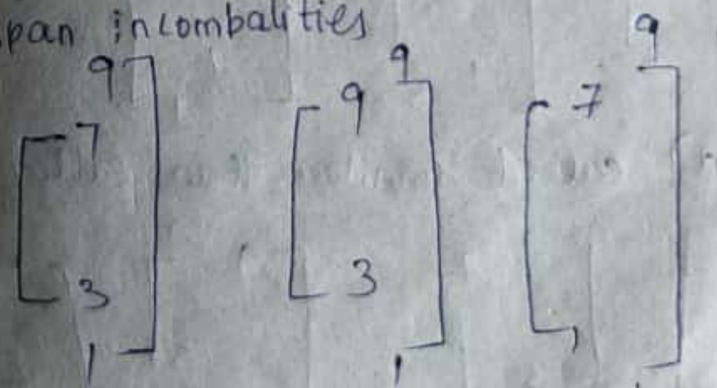
R] BR Bottom





Span Compatibility

Figure 4-18 shows three possibly harmless span incombalities

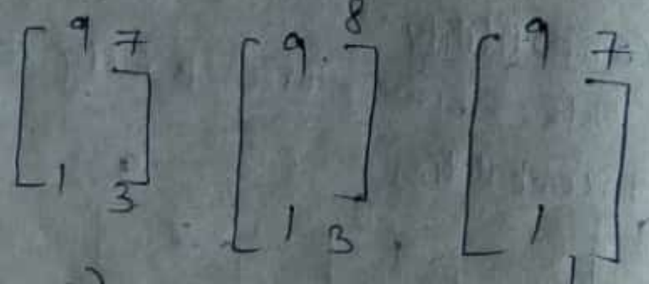


Harmless Range/Domain span incompati-bility bug (caller span is smaller than called).

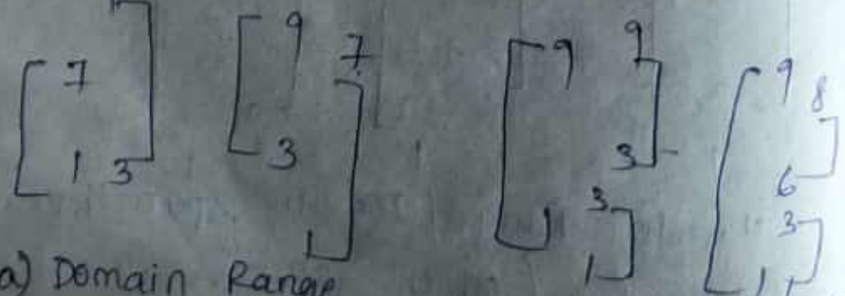
→ In all cases, the caller's range is a subset of the called's domain. That's not necessarily a bug.

→ The routine is used by many callers - some require values inside a range and some don't. This kind of span incompatibility is a bug only if the caller expects the called routine to validate the called number for the caller.

→ Figure 4-19 a shows the opposite situation, in which the called routine's domain has a smaller span than the caller expects. All of these examples are buggy.



a) called smaller than caller.



a) Domain Range Mismatch (c) Holes in the called domain



Figure 4-19: Buggy Range/Domain mismatches

Figure 4-19b the ranges and domains don't line up, hence, good values are rejected, bad values are accepted, and if the called routine isn't robust enough. we have crashes.

- Figure 4-19c combines these notions to show various ways we can have holes in the domain: these all are probably buggy.

Interface / Domain Compatibility Testing

→ For interface testing, bugs are more likely to concern single variables rather than peculiar combinations of two or more variables.

→ Test every input variable independently of other input variables to confirm compatibility of the caller's range and the called routine's domain span and closure of every domain defined for that variable.

→ There are two boundaries to test and

it's a one-dimensional domain ; there it requires one on and one off point per boundary or a total of two on points and two off points for the closure (COOOOI). pick off points appropriate to the

→ start with the called routine's domains and generate test points in accordance to the domain-testing strategy used for that routine in component testing.

→ unless you're a mathematical whiz you won't be able to do this without tools for more than one variable at a time.

Date 30/1/26

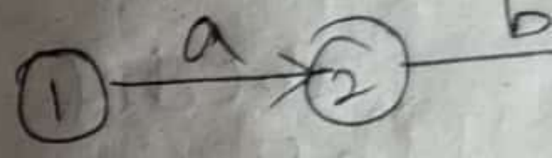
UNIT-

Paths, Pa

Regul

→ The path is a path segments as a name

→ The path is of all path



The path pr

means concat

→ In the fig segments a, b to 5 can be

→ That is ex

→ This abcd product

→ If A, B ar

A = abc B

is AB = ab

is abcdef.

For Ex:- A